

導航 (Navigation)

台灣有許多高山，三千公尺以上的山頭就有超過 250 座。阿明計畫要在中央山脈建構空中滑索網路以支援在偏鄉間傳遞包裹。

在阿明的計畫裡，每一個網路由 N 個節點組成，節點編號由 0 至 $N - 1$ 。這些節點裡恰好有 $K = 6$ 個是供電站，而其他 $N - K$ 個節點代表鄉村。這些節點被 $N - 1$ 條雙向的纜線給連繫起來，纜線編號由 0 至 $N - 2$ 。在這區間內 $0 \leq i \leq N - 2$ 的每個 i 值，第 i 條纜線連接了節點 $U[i]$ 和節點 $V[i]$ 。每條纜線連接兩個相異的節點，且每一對節點只能被至多一條纜線給連接。保證任兩個節點可以透過纜線們來移動。

節點 a 和節點 b 之間的距離用 $d(a, b)$ 表示，並且定義如下。

- 如果 $a = b$ ，則 $d(a, b) = 0$ 。
- 否則， $d(a, b)$ 等於最少需要使用多少纜線能從節點 a 移動到節點 b 。

在一個網路中，如果每個節點都被恰好一條或是至少 δ 條纜線連接，我們說這個網路的分支度至少有 δ 。阿明知道這個網路的分支度至少有 B ，且 B 是一個正整數。

阿明準備了一百種不一樣的纜線，型號由 0, 1 至 99。在上述網路中的每一條纜線，型號都會是這裡面的其中一種。

機器人們會在空中滑索纜線之間移動、實現遞送包裹的任務。阿明想要安裝導航程式幫助機器人們回到供電站來幫自己充電。因為機器人有非常少的記憶體，所以在設計導航程式時，機器人的導航只能依據以下這些資訊：供電站的數量 $K = 6$ 、保證的分支度 B 、以及連接到機器人目前所在位置的所有纜線的型號。

當一個機器人想要在鄉村節點 u 做導航，這個節點被兩條或更多條的纜線連接，這機器人首先會將連接到 u 的纜線，以任意的順序看過一遍。導航程式會提供一個清單列表，依照剛剛的順序說明纜線的型號。導航程式需要針對這些纜線的每一條，如果機器人沿著這條纜線走，計算出有多少個供電站，它到機器人的距離會縮短。

精確地說，令 $v_1, \dots, v_k$ ($k \geq 2$) 為透過一條纜線直接連接到節點 u 的所有節點、令 c_i ($1 \leq i \leq k$) 為連接節點 u 和節點 v_i 的型號。導航程式會提供 K, B , 和型號清單 $c_1, c_2, \dots, c_k$ 。對於每一個 k ($1 \leq i \leq k$)，導航程式需要計算出有幾個充電節點 p 滿足 $d(v_i, p) < d(u, p)$ 。

你的任務是設計出一個策略來安排纜線型號、設計導航程式。你的解法的分數取決於你使用的纜線型號有幾種 (細節請看子任務和分數小節)。

實作細節 (Implementation details)

你需要實作以下兩個函式，一個用來指定纜線型號、另一個用於機器人的導航程式。

用來**指定纜線型號**的函式如下：

```
std::vector<int> construct_network(int N, int K, int B,
                                   std::vector<int> U, std::vector<int> V,
                                   std::vector<int> P)
```

- N : 節點數量。
- K : 充電節點數量。
- B : 保證的最小分支度。
- U, V : 長度為 $N - 1$ 的陣列，用於描述纜線的連接。
- P : 長度為 $K = 6$ 的陣列，用於描述充電節點。
- 在每一筆測試資料中，這函式被呼叫**至多 10 000 次**。

這函式應該要回傳一個陣列 T ，其長度為 $N - 1$ ：

- 第 i 條纜線型號 $T[i]$ ，被用於連接節點 $U[i]$ 和節點 $V[i]$ 。
- $T[i]$ 的數值需要滿足 $0 \leq T[i] < 100$ 。

用於**機器人的導航程式**的函式如下：

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K : 充電節點數量。
- B : 保證的最小分支度。
- C : 列舉連接到某節點的所有纜線型號，按照任意一個順序列舉。這節點是某個鄉村節點，被至少兩條纜線給連接。
- 保證 C 的長度至少有 B 。
- 在每一筆測試資料中，這函式被呼叫至多 100 000 次。
- 這函式或許不能依賴一開始給定的網路結構；舉例來說，評分程式或許會在同一筆測資中所建構的不同網路中，重新安排呼叫這個函式的順序。

這函式應該要回傳一個陣列 D ：

- 陣列 D 的長度應該要和陣列 C 一樣。
- $D[i]$ 代表這個數字：若機器人沿著對應到 $C[i]$ 的電纜走，有幾個充電站到機器人的距離會減少。

留意在評分系統裡，`construct_network` 和 `navigate` 這兩個函式，是被 **兩個不同的行程 (process)** 給呼叫。

條件限制 (Constraints)

- $K = 6$.
- $K < N \leq 100\,000$.
- 在一筆測試資料中，`construct_network` 函式的所有呼叫中， N 值的累積總和不會超過 100 000。
- $0 \leq U[i] < V[i] < N$ 當 $0 \leq i \leq N - 2$.
- 任兩個節點，都可以利用纜線們來從一個節點移動到另外一個節點。
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- 每一個陣列 C 都是以下的一種排列：連接到某個節點的所有纜線們的型號，這節點是某個鄉村節點且至少被兩條纜線所連接。
- 在一筆測試資料中，`navigate` 函式的所有呼叫中，陣列 C 長度的累積總和不會超過 200 000。

子任務與得分 (Subtasks and Scoring)

子任務	分數	額外限制
1	6	$B = 2, N = 7$.
2	14	$B = 2, U[i] = i, V[i] = i + 1$ 對於每個 i 當 $0 \leq i < N - 1$.
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

在任何一筆測試資料中，如果下面的至少一個條件成立，那麼你的解答的分數在這筆測資的成績就會是 0 (在 CMS 中被回報為 `Output isn't correct`)：

- 任何對 `construct_network` 函式的呼叫，得到的回傳值不合法。
- 任何對 `navigate` 函式的呼叫，得到的回傳值不合法。

否則，令 S 為最小的整數滿足所有對 `construct_network` 函式呼叫所得到的回傳陣列 T 中的所有數值都小於它。換句話說， S 為最小的整數滿足所建構的網路只有使用纜線型號 $0, 1, \dots, S - 1$ 。

在每一筆測資中，你得到的分數取決於 S ，如下所示：

條件	子任務 1	子任務 2	子任務 3 與 4	子任務 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2 \log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

特別留意，在子任務 1, 2 以及 5，你會得到全部的分數如果 $S \leq 5$ ，在子任務 3 以及 4，你會得到全部的分數如果 $S \leq 4$ 。

範例 (Example)

考慮以下場景 $N = 9$ 、 $K = 6$ 以及 $B = 2$ 。

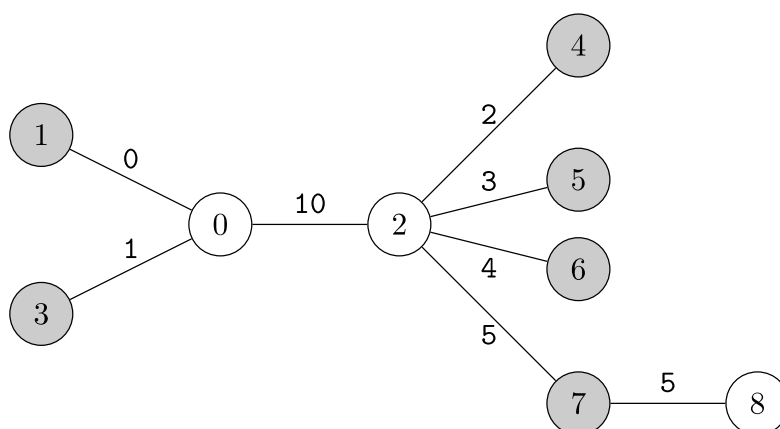
網路的描述如：陣列 $U = [0, 0, 0, 2, 2, 2, 2, 7]$ 以及陣列 $V = [1, 2, 3, 4, 5, 6, 7, 8]$ 充電節點在 $P = [1, 3, 4, 5, 6, 7]$ 。在第一個行程，呼叫

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                  [1, 2, 3, 4, 5, 6, 7, 8],
                  [1, 3, 4, 5, 6, 7])
```

如果阿明建立的網路回傳了：

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

所建構的網路如下繪製：



接著，在第二個行程，如果機器人需要在節點 0 上導航。節點 0 連接到節點 1, 2 以及 3。如果機器人讀取纜線型號是按照這個順序，會進行下面的呼叫：

```
navigate(6, 2, [0, 10, 1])
```

根據網路結構：

- 第一個充電節點（節點 1）離自己的距離最短。例如 $0 = d(1,1) < 1 = d(0,1) < 2 = d(2,1) = d(3,1)$ 。
- 第二個充電節點（節點 3）離自己的距離最短。例如 $0 = d(3,3) < 1 = d(0,3) < 2 = d(1,3) = d(2,3)$ 。
- 其他充電節點（節點 4, 5, 6 以及 7）到節點 2 的距離較短。例如 $1 = d(2,u) < 2 = d(0,u) < 3 = d(1,u) = d(3,u)$ 當 $u = 4, 5, 6$, 或 7。

若沿著纜線 (0,1), (0,2), 和 (0,3) 走，距離減少的充電節點數量分別是 1, 4 和 1。因此，這函式應該要回傳：

```
[1, 4, 1]
```

留意到 `navigate` 函式要能夠支援陣列 C 是以不同的排列方式出現。舉例來說，`navigate(6, 2, [1, 0, 10])` 也是一種在節點 0 上的可能呼叫。這函式應該要回傳 `[1, 1, 4]`。

如果機器人需要在節點 2 上導航。執行呼叫如下：

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

這函式應該要回傳：

```
[2, 1, 1, 1, 1]
```

在這個網路中，`navigate` 函式從頭到尾都不會在其他節點上呼叫，因為：

- 節點 1, 3, 4, 5, 6 以及 7 都是充電節點，且
- 節點 8 只連接一條纜線。

在筆測試資料中，被使用的纜線型號是 0, 1, 2, 3, 4, 5, 以及 10。所以 $S = 11$ 會當作評分依據。

這题目的附加檔案中，包含兩筆其他的輸入和輸出可用於範例評分程式的測試上。

範例評分程式 (Sample Grader)

範例評分程式會在同一行程 (process) 裡呼叫 `construct_network` 和 `navigate` 兩個函式。此外，當範例評分程式呼叫 `navigate` 函式，纜線型號的列舉順序如同他們所對應的纜線在輸入裡面的順序。正式評分程式和範例評分程式在上面兩種行為有所不同。

輸入格式：

```
T
(scenario 0)
(scenario 1)
...
(scenario T-1)
```

每一個場景以下面的格式呈現：

```
N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

在每一個場景，範例評分程式會呼叫 `navigate` 函式 Q 次，第 i 次呼叫使用了節點 $X[i - 1]$ 的資訊。留意到範例評分程式支援 K 設定成和 6 不一樣的整數。

輸出格式：

如果 `construct_network` 或 `navigate` 函式的任何回傳陣列是不合法的，範例評分程式會終止並且列出原因。否則，範例評分程式會輸出 `navigate` 函式的回傳陣列 D 在同一行。

當所有場景都被處理後，範例評分正式會印出一行到標準除錯 (standard error)：

```
S = S'
```

其中 S' 是最小的整數滿足：在這筆測資中，所有 `construct_network` 函式的回傳陣列 T 裡面的數值都小於它。